00 Pattern Finder

Start every question here. Circle the words in the question → find them below → go to the tab. START HERE

1
Circle the trigger words
"optional add-ons", "only one", "states", "in order"...

2
Match them below
Each trigger points to one pattern and its tab number.

3
Open the tab → WRITE page
Copy the blocks in order: **Name → Why → Diagram → Code → Challenges**.

IF THE QUESTION SAYS...	IT'S...	TAB
"only one instance", "single global tax calculator / logger / ID generator", "run by only one user", "new object every time ⇒ duplicates"	C **Singleton**	→ 01
"maximum N" — 5 batsmen, 3 printers, N gates	C **Multiton** (extended Singleton)	→ 01
"create different types of X depending on user request", "too many if-else for creation", "extensible product framework"	C **Factory Method**	→ 02
"one simple interface by internally calling A, B, C", "combine air + hotel + taxi", "hide complexity"	S **Facade**	→ 03
"legacy / vendor / can't modify", "new device has a different method", "third-party services with different signatures"	S **Adapter**	→ 04
"load only a couple of pages", "should not access directly", "restricted / permissions", "authorization", "cache"	S **Proxy**	→ 05
"optional add-ons / extras", "without subclasses for every combination", "price = base + selections"	S **Decorator**	→ 06
"same overall process / common workflow, however step X differs", "enforce the procedure"	B **Template Method**	→ 07
"checks in order, easy to extend", "levels of support / approval", "sub-filters", "priorities", "exception handling"	B **Chain of Responsibility**	→ 08
"operates in states", "transitions to", "behaviour depends on current state", "draw a state diagram"	B **State**	→ 09
"microservices", "scalability, fault isolation, continuous deployment", "data consistency", "security"	A **Microservices kit**	→ 11
"High Availability", "Disaster Recovery", "configurability", "use of interfaces / abstract classes", SOLID	A **Answer bank**	→ 12

Still unsure? Ask one question.

Is it about **making objects**?	Creational one → Singleton · types → Factory
Is it about **wrapping / connecting** objects?	Structural see the 4-wrapper test →
Is it about **who does what, in what order**?	Behavioural fixed steps → Template · pass along → CoR · states → State

The 4-wrapper test

all four "wrap" something

Adapter	**changes** the interface (translator 🔌)
Facade	**simplifies** many objects into one (waiter)
Proxy	same interface, **controls** access (guard)
Decorator	same interface, **adds** features, stackable (toppings)

One scenario, many patterns

2025 style — every part gets its own pattern

2025 Q1 Kiosk	states → **State 09** · add-ons → **Decorator 06** · one tax calculator → **Singleton 01**
2025 Q3 Library	checks in order → **CoR 08** · book types → **Factory 02** · rare books → **Proxy 05** · optional services → **Decorator 06**
2025 Q4 E-commerce	common payment steps → **Template 07** · validations → **CoR 08** · Order IDs → **Singleton 01** · payment objects → **Factory 02**
2022 Q1 Travel	combine air + hotel + taxi → **Facade 03** · each third-party API → **Adapter 04**

What will come in 2026

Built from 4 past papers (2022–25) and the lecturer's 7 in-class quizzes.

READ ONCE, BEFORE THE EXAM

THE PAPER
4 Qs × 25
answer **all** · 150 min

YOUR CLOCK
35 min / Q
≈ **1.4 min per mark** · 10 min spare

ALWAYS
State assumptions
The paper says so. One line at the top.

FORECAST

Likely questions

how sure I am

#	PREDICTION	EVIDENCE	CONFIDENCE	TAB
1	**Q2 = Microservices** for a new domain, same 5 sub-questions. Domains not used yet: food delivery, banking/ATM, LMS, hospital, ride-hailing.	2023 2024 2025 word-for-word	very high	**11**
2	**"Identify the patterns for (a) (b) (c) (d)"** in one scenario, with reasons. Then short code for one of them.	2025 Q1, Q3, Q4 all did this	very high	**00**
3	**Write an interface with 2 methods + one concrete class** (Decorator) — e.g. `getCost()`, `getDescription()`.	2024 Q1.2–1.3 2025 Q1.2–1.3	high	**06**
4	**Singleton** for "one global X" — explain + how it plugs into the flow.	every paper, 9 times total	high	**01**
5	**Abstract class with a template method** + one concrete subclass.	2025 Q4.2 Quiz 6	high	**07**
6	**State** with a state diagram (Idle → ... → Completed style).	2024 Q3 2025 Q1 Quiz 1	high	**09**
7	**Adapter / Facade comeback.** New slides + Quiz 1 & 3 this year, but missing from papers since 2023 → overdue.	Quiz 1 Quiz 3	medium	**03 04**
8	**SOLID** — "which principle does this follow?" or "explain the use of interfaces". New lecture, never examined.	Lecture 12 (new)	medium	**10 12**
9	**Challenges → mechanism, HA & DR** (older style, may return as a 4–5 mark part).	2022 2023	medium-low	**12**

The lecturer's question recipe

every quiz follows it

① Why this pattern?
2–4 marks. Name + category + 2–3 reasons tied to the scenario.

② Draw it
4–8 marks. State diagram and/or UML class diagram.

③ Java code
3–6 marks. Interface / abstract class / context class.

④ The twist
3–5 marks. A second pattern added at the end (Quiz 1 → Adapter, Quiz 3 → Facade, Quiz 6 → Decorator, Quiz 7 → Template).

Questions that came back almost word-for-word

REPEATED QUESTION	SEEN	ANSWER
"LMS — load only a couple of pages rather than the entire document"	2022 Q4.4 2023 Q4.4	Virtual **Proxy** → 05
"Define a Coffee / CheckoutItem interface with getCost() and getDescription()"	2024 Q1.2 2025 Q1.2	**Decorator** → 06
"Salary components differ for executive / daily / temporary staff"	2022 Q2 2023 Q1	**Template** + Singleton → 07
"Exception handling with different priorities / types"	2023 Q4.3 2024 Q4.3	**CoR** → 08
"Different authorization options / levels"	2023 Q4.5 2024 Q4.5	Protection **Proxy** (+CoR) → 05
"Logger — all logs to the same file" / "implement logging"	2023 Q3.4 2024 Q1.5	**Singleton** → 01
"Extend this design for High Availability and Disaster	2022 Q3.5 2023 Q3.5	Answer bank → 12

06 Decorator

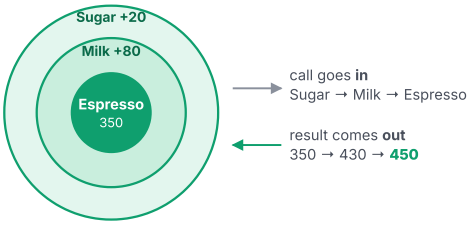
Add optional extras by **wrapping** the object — no subclass per combination.

STRUCTURAL · UNDERSTAND

SPOT IT — THE QUESTION SAYS...

optional add-ons / extras / toppings · without subclasses for every combination · price depends on base + selected additions · instead of modifying the class · students may optionally add · bloated class with optional features

The idea in one picture



Why not just use subclasses?

Every combination needs its own class (`EspressoWithMilkAndSugar` ×) = **class explosion**:

3 add-ons → **8** · 10 → **1,024** · 15 → **32,768** (2^n)

Decorator builds the combination **at runtime** with objects, not at compile time with classes.

The 4 parts → rename for your question

naming them = marks

	COMPONENT «INTERFACE» the shared type	CONCRETE COMPONENT the plain base	CONCRETE DECORATORS each adds one extra, then delegates
Coffee 24	`Coffee`	`BasicCoffee`	`Milk` · `Sugar` · `Chocolate`
Kiosk 25	`CheckoutItem`	`GroceryItem(name, price)`	`GiftWrap` · `EcoBag` · `ExpressPacking`
Profile Q5	`UserProfile`	`BasicUserProfile`	`PremiumBadge` · `TwoFactorAuth` · `Theme`
Library 25	`BorrowTransaction`	`BasicBorrow`	`ExtendedPeriod` · `SmsNotification` · `EmailReceipt`
Cake 22	`Cake` price + days	`PlainCake`	`Fondant` · `Colour` (each adds price and delivery days)
Course Q6	`Course`	`ProgrammingCourse`	`AIAssistant` · `PremiumMaterials` · `MentorSupport`

The 4th part, the abstract **Decorator**, is always `<Component>Decorator` — it is-a and has-a Component.

Don't confuse it

- **vs Proxy** — same interface, but Proxy **controls** (1 layer); Decorator **adds** (many layers).
- **vs Adapter** — Adapter **changes** the interface; Decorator keeps it.
- **vs Inheritance** — compile time, rigid vs runtime, flexible.

Common mistakes

lose marks

- Decorator does **not implement** the Component interface
- Decorator creates its own base instead of **receiving it in the constructor**
- Forgets to **delegate** → loses earlier add-ons
- Draws inheritance instead of **has-a (◇)** to the Component

Every time it was asked

7 times · ≈ 70 marks total

WHERE	WHAT THEY ASKED	TAKEAWAY
2024 Q1.1–1.4 **20m**	Coffee + milk/sugar/chocolate: pattern · Coffee interface · BasicCoffee · different profit	Profit → `ProfitMargin` decorator **W4**
2025 Q1 **11m**	Kiosk add-ons: CheckoutItem interface · GroceryItem class	Same code, new names **W3**
2023 Q3.1–3.2 **10m**	Coffee flavours (Irish, Nitro): pattern + simple class diagram	Irish/Nitro = bases **W2**
Quiz 5 **18m**	Profile badge/2FA/theme: how applied · classes · wrapping code · challenges	Challenges **W5**
2022 Q4.1 · 2025 Q3.1d · Quiz 6	Cake extras · library optional services · course extras (2–5m each)	Name + 2 reasons **W1**

Takeaway: the question gives you the method names — use them exactly. The code is the same every year.

06 Decorator — Write

Copy-ready blocks. Swap the **bold names** for the ones in your question.

STRUCTURAL · WRITE

W1 Name it + why

2–6 marks

> The **Decorator** design pattern (**structural**) is suitable. It attaches additional responsibilities to an object **dynamically** by wrapping it, giving a flexible alternative to subclassing.

Reasons (pick 2–4):

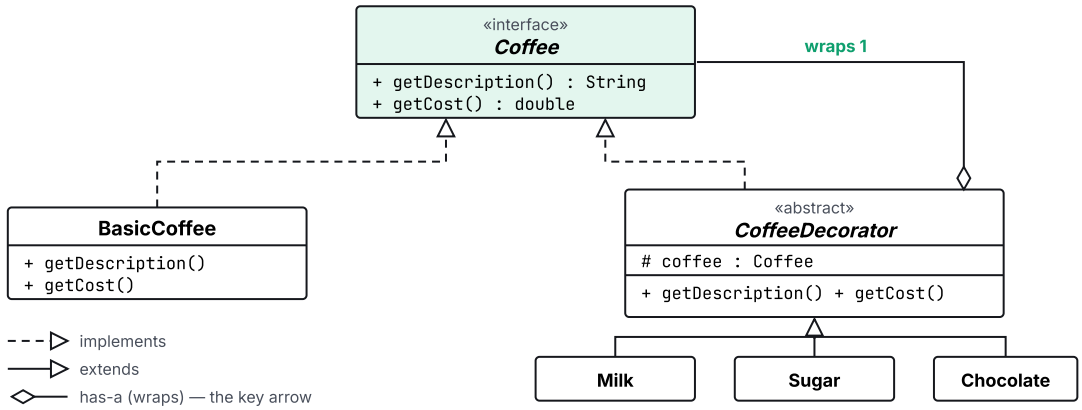
- **No class explosion** — n add-ons would need 2^n subclasses; here, one class per add-on.
- **Runtime flexibility** — the customer chooses add-ons while ordering; any mix, any order.
- **Open/Closed** — a new add-on is a new class; the base and existing add-ons stay unchanged.
- **Single Responsibility** — each decorator adds one thing (price + description).

Map it to the scenario (where the marks are):

- Component = **Coffee** «interface»
- Concrete component = **BasicCoffee**
- Decorator = **CoffeeDecorator** «abstract»
- Concrete decorators = **Milk, Sugar, Chocolate**

W2 Class diagram

5–8 marks · draw exactly this shape



W3 · JAVA — COPY IN THIS ORDER · 5 + 5 MARKS

```
// 1. COMPONENT — the interface they ask you to "define"
public interface Coffee {
    String getDescription();
    double getCost();
}
// 2. CONCRETE COMPONENT — the "basic" class
public class BasicCoffee implements Coffee {
    public String getDescription() { return "Basic coffee"; }
    public double getCost() { return 200.0; }
}
// 3. ABSTRACT DECORATOR — is-a AND has-a Coffee
public abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;              // wrapped
    public CoffeeDecorator(Coffee c) { coffee = c; }
    public String getDescription() { return coffee.getDescription(); }
    public double getCost() { return coffee.getCost(); }
}
// 4. CONCRETE DECORATOR — add, then delegate
public class Milk extends CoffeeDecorator {
    public Milk(Coffee c) { super(c); }
    public String getDescription() {
        return coffee.getDescription() + ", Milk"; }
    public double getCost() { return coffee.getCost() + 80; }
}   // Sugar (+20), Chocolate (+120): same shape

// 5. CLIENT — wrap at runtime
Coffee c = new BasicCoffee();
c = new Milk(c);
c = new Sugar(c);
System.out.println(c.getDescription()); // Basic coffee, Milk, Sugar
System.out.println(c.getCost());        // 300.0
```

W4 "Different profit?"

5m

A — add `getProfit()` to the interface; each class adds its margin.
B (better, OCP) — a new decorator; nothing existing changes:

```
class ProfitMargin
    extends CoffeeDecorator {
  double rate;
  ProfitMargin(Coffee c, double r)
    { super(c); rate = r; }
  public double getCost() {
    return coffee.getCost()
           * (1 + rate); }
}
```

W5 Challenges → fix

3–4m

- **Complexity** (many small classes) → one decorator = one job; a builder assembles common combos.
- **Debugging** through layers → log in each decorator; unit-test each alone.
- **Order matters** (discount→tax ≠ tax→discount) → fix the wrapping order in the builder.
- **Performance** (extra objects) → negligible for short chains; cache the cost.
- **Readability** (`new A(new B(x))`) → builder, clear names, docs.

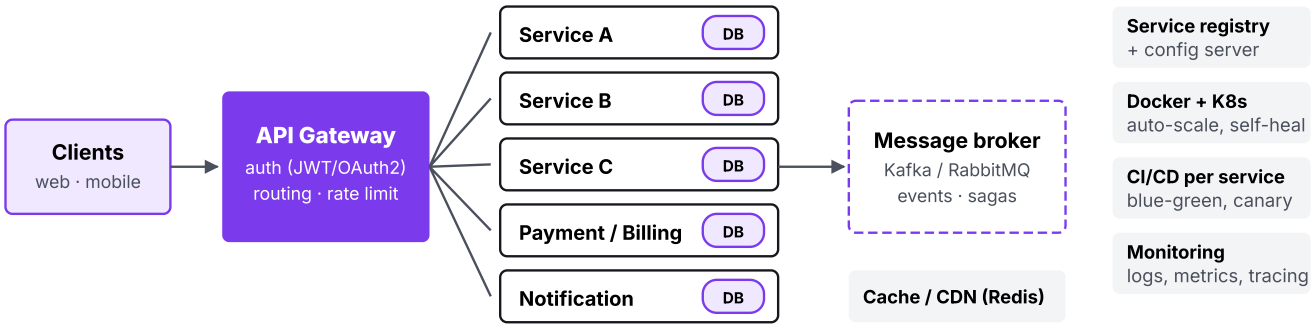
11 Q2 Microservices Kit

Same 5 questions every year — only the business changes. Fill the blanks with your domain's services.

25 MARKS · EVERY YEAR

1 Draw the scenario

4–5 marks · copy this skeleton, rename the middle column



IF THE DOMAIN IS...	SERVICE A · B · C (+ PAYMENT, NOTIFICATION)
Food delivery	User · Restaurant/Menu · Order · Delivery/Rider tracking
Banking	Customer · Account · Transaction · Loan · Fraud detection
LMS	User · Course/Content · Assessment/Grading · Enrolment
Hospital	Patient · Appointment · Doctor/Staff · Pharmacy · Lab results
Ride-hailing	Rider · Driver · Trip/Matching · Pricing · Location
Already asked	Supermarket (Purchasing, Sales/POS, Warehouse, Supplier, Branch) · Library (Book, User, Borrowing, Catalog Search) · Streaming (Profile, Content+CDN, Recommendation ML, Analytics)

2 Scalability · fault isolation · CD

5m

- **Scalability** — each service scales **independently** (horizontal pods, auto-scale on CPU/requests). Scale the busy one (e.g. *Content / Search*), not Billing. Stateless services; cache/CDN.
- **Fault isolation** — own process + own DB; **circuit breaker**, timeouts, retries with back-off, bulkheads; **graceful degradation** (Recommendations down → show popular list).
- **Continuous deployment** — own repo + CI/CD pipeline per service, automated tests, Docker images, **blue-green / canary**, versioned APIs; one team per service.

3 Propose the architecture

4–5m

- One service per **business capability** (list yours from the table) — own codebase, own DB, own deployment.
- **API Gateway** as single entry; **service registry** for discovery.
- Sync: REST/gRPC. Async: events through the **message broker**.
- Containers on **Kubernetes**; central config, logging, monitoring.

4 Data consistency

4–5m

- **Database per service** ⇒ no shared transactions ⇒ **eventual consistency**.
- **Saga pattern** — local transactions + **compensating actions** (e.g. payment fails → cancel borrow). Choreography (events) or orchestration.
- **Transactional outbox** + idempotent consumers; **CQRS** for read models (search).
- Two-phase commit only where strong consistency is critical (rare).

5 Security

4–5m

- **Users**: OAuth2 / OpenID Connect at the gateway, **JWT** tokens, role-based access (RBAC).
- **Service-to-service**: **mutual TLS**, service mesh, short-lived tokens.
- **Data**: HTTPS everywhere, encrypt at rest (PCI-DSS for payments), secrets vault, least-privilege DB access.
- Rate limiting, input validation, WAF, audit logging.

2023 extra — "Why same hardware/config for each platform?" Consistent environments (containers / infrastructure-as-code) ⇒ predictable deployment, scaling and failover; any node runs any service; easier capacity planning, monitoring, troubleshooting and DR rebuild. Exception: GPU nodes for ML services.